DOCUMENT

15

Demo Video Script

A timed, rehearsed script for the 8 to 10 minute recording

Target length: **9 minutes 30 seconds** (the assignment asks for 8 to 10). Format: one MP4. Something must run live on screen; a read-through of files is not a demo, so five of the nine segments below run something live.

0. What the viewer must come away with

1. The feature works: limits by role, two distinct approvers, refusals that change nothing.
2. The tests can fail: a deliberate one-character bug turns the suite red.
3. The tests are good: mutation score 63.9% before, 100% after.
4. The AI work was controlled: context file, spec, agents, hooks, a log, and a review where every finding got a verdict.

1. Timeline

Time	Segment	Live?	Screen
0:00 - 0:45	1. Introduction	no	Repo in the editor, README rules table
0:45 - 1:45	2. Context, skills, agents, hooks	no (short)	`AGENTS.md`, `.claude/` tree
1:45 - 2:30	3. Specification	no (short)	`specs/001-invoice-approval-limits/`
2:30 - 4:00	4. Unit tests, **live**	**yes**	Terminal: pytest, then the deliberate break
4:00 - 5:30	5. Mutation testing, **live**	**yes**	Terminal: mutmut script in WSL, before/after text
5:30 - 6:30	6. Playwright, **live**	**yes**	Headed browser
6:30 - 7:45	7. The feature in the UI, **live**	**yes**	Browser: Manager, Director, second Director
7:45 - 8:45	8. PR review and verdicts	no	`docs/pr-review.md` or the PR page
8:45 - 9:30	9. AI log and wrap-up	**yes** (tail the log)	Terminal and `ASSIGNMENT.md`

Speak at a steady pace: about 150 words a minute. The narration below is written to fit its slot; do not read it word for word.

1a. A ready-made draft

`python scripts/make_demo_video.py` records this same storyline automatically in about ten minutes of real time and writes `docs/video/demo-narrated.mp4` (synthetic voice, with captions) and `docs/video/demo-captions-only.mp4` (silent, so you can add your own voice). Every command in it runs live while recorded: the test suite, the one-character break and its restoration, the mutation run in WSL, the Playwright test and

the log query. The UI walkthrough drives the real app on a fresh server. The narration states that the recording was produced by a script. Use it as a draft, or record your own version with the steps below.

2. Before you press record (checklist)

1. Terminal at the repo root, font size large (at least 16 pt), light or dark theme with good contrast. Two terminals: **PowerShell** for everything and one **browser** window.
2. Close notifications, Teams chat and anything with personal data. Hide the OneDrive path if you can (shorten the prompt: `function prompt { "PS> " }`).
3. Nothing running on port 5000. Check: `Get-NetTCPConnection -LocalPort 5000 -ErrorAction SilentlyContinue` returns nothing.
4. Fresh state: the server resets on restart. The manual walkthrough (segment 7) needs INV-2001 and INV-2002 untouched, so start the server fresh before recording. The Playwright test (segment 6) makes and removes its own invoice, so it does not change them.
5. Dry run once: `python -m pytest backend/tests -q`, `cd e2e; npm test`, and the mutation script (see segment 5). All three must be green.
6. If you have committed the work: `git status` is clean, so the deliberate break in segment 4 can be undone with `git restore`. If you have not committed: copy the file first (segment 4 shows how).
7. Tabs/files to have open: `README.md`, `AGENTS.md`, `specs/001-invoice-approval-limits/spec.md`, `docs/mutation/baseline.txt`, `docs/mutation/after.txt`, `docs/pr-review.md`, `logs/ai-interactions.jsonl`, and the PR page if it exists.
8. Record with Teams (start a meeting with only yourself, press Record) or any recorder that outputs MP4. Record the whole screen at 1080p. Do a 10-second test recording and check the audio.

3. The script

Segment 1: Introduction (0:00 - 0:45)

Show: the README, scrolling to the "Approval limits by role" table.

Say: "This is my AI-in-SDLC assignment. The feature is approval limits by role for an invoice approval service. A Manager can approve up to fifty thousand, a Director up to five hundred thousand, and anything above that needs two different approvers where the second must be a Director. A refused attempt must say why and change nothing. The original repo is C# and I have no .NET SDK, so I ported the backend to Python with FastAPI, kept the same layers, and kept the original tests green before I touched the feature. I used Claude Code with Spec Kit. In this video I will run the tests, the mutation tests and the end-to-end test live."

Segment 2: Context, skills, agents, hooks (0:45 - 1:45)

Show: `AGENTS.md` (the rules table and "Review expectations"), then the `.claude` folder tree in the explorer.

Say: "First, context. AGENTS.md is what an agent reads when it arrives cold. It holds the approval rules, the boundaries to test, the layering rule that business logic lives only in the domain, the test command, and my review expectations: a refusal test must check the reason and that the invoice is unchanged. In

`.claude` there are three skills: approval-rules, review-findings and run-checks. There are five agents: a spec-writer, an implementer that does one step at a time, a test-reviewer, a pr-reviewer, and a mutation-analyst. The hooks write every prompt, tool call, sub-agent result and test run into one log file in the same format."

Optional 10 seconds, strong: say "I also tested whether the context file matters" and show the table in `ASSIGNMENT.md` section 7 (cold agent versus agent with AGENTS.md).

Segment 3: Specification (1:45 - 2:30)

Show: `specs/001-invoice-approval-limits/spec.md` Clarifications, then `tasks.md`.

Say: "The specification comes from Spec Kit. The Clarifications section is where I resolved the ambiguities: the limits are inclusive, so exactly fifty thousand is allowed for a Manager; the first approver above five hundred thousand can be a Manager or a Director; and 'the same user' means the same username ignoring case and spaces. Tasks are split into two stories, single approver first, then the two-approver rule, and I implemented them in that order."

Segment 4: Unit tests, live (2:30 - 4:00)

Run:

```
python -m pytest backend/tests -q
python -m pytest backend/tests/test_approval_policy.py -v -k "same_user or second_approver"
```

Say (while the first command runs, about 1 second): "One hundred thirty-four tests, all green in a few seconds. They are split into the pure policy rules, the service with a fake repository, and the HTTP API."

Show the second command's output and say: "These are the two negative rules the assignment warns about: the same user can never be both approvers, and a Manager can never be the second approver. Let me open one."

Show: `assert_refused_and_unchanged` in `backend/tests/test_approval_service.py` (search for it).

Say: "Every refusal test uses this helper. It deep-copies the invoice, expects the refusal, asserts the exact reason text, asserts the invoice equals the copy, and asserts the repository saw no save. A test that only checks the exception type would pass even if the code half-applied the approval."

Now prove the tests can fail. Open `backend/src/invoice_app/domain/approval_policy.py`, the line `needs_director = self._limits.manager < attempt.amount <= self._limits.director` in `ManagerLimitRule`.

If **committed**:

```
# edit: change the first < to <= and save, then
python -m pytest backend/tests -q -x
git restore backend/src/invoice_app/domain/approval_policy.py
python -m pytest backend/tests -q
```

If **not committed** (back up first, off camera):

```
Copy-Item backend\src\invoice_app\domain\approval_policy.py $env:TEMP\approval_policy.bak
# edit: change the first < to <= and save, then
python -m pytest backend/tests -q -x
Copy-Item $env:TEMP\approval_policy.bak backend\src\invoice_app\domain\approval_policy.py -Force
python -m pytest backend/tests -q
```

Say: "I change one character: less-than-or-equal becomes less-than at the Manager limit. Exactly fifty thousand would now be refused. The boundary test fails immediately: `test_manager_may_approve_up_to_and_including_the_manager_limit` with 50000. I restore the file and everything is green again. That is the point of testing one cent either side of every limit."

(Rehearsed: this produces "1 failed, 38 passed" with `-x`, then 177 passed after the restore.)

Segment 5: Mutation testing, live (4:00 - 5:30)

Show first: `docs/mutation/baseline.txt` (63.9%), then run:

```
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh demo
```

(If Git Bash is used instead of PowerShell, prefix with `MSYS_NO_PATHCONV=1` and use the full `/mnt/c/...` path to the script. The run takes about 20 seconds end to end.)

Say before running: "Passing tests are not proof that tests are good. Mutation testing changes the code in small ways, like flipping a comparison, and checks that a test notices. Before the feature, with the seven original tests, the score was sixty-three point nine percent: twenty-three of thirty-six mutants killed. I will run it live now on the finished code. It runs under WSL because the tool needs fork."

While it runs (about 20 seconds): "There are eighty-six mutants, because the new policy module is now in scope. Note that the comparison is fair in scope, but the mutant count is higher."

When it finishes, show the printed block and say: "Eighty-six of eighty-six killed: one hundred percent. It did not start there. My first pass after the feature was eighty-one percent, with sixteen survivors. They were all missing tests, none equivalent: a fake repository whose update did nothing, an unasserted approval timestamp, naive timestamps and unasserted error messages. I added tests for each. The caveat: this measures the domain unit tests only, not the API layer."

Afterwards: `Remove-Item docs\mutation\demo.txt` (so the demo file does not pollute the repo).

Segment 6: Playwright end-to-end, live (5:30 - 6:30)

Make sure no server is running, then:

```
cd e2e
npx playwright test --headed
```

(`npm run test:headed` does the same.)

Say before running: "The one required end-to-end test: a rejected over-limit approval through the real UI. It uses page objects and role-based locators, no CSS selectors."

While the browser runs (about 6 seconds): "It signs in as the demo Manager, opens Approvals, clicks Approve on the seventy-five thousand invoice, expects the exact refusal message on screen, expects the invoice to still be listed as pending, and then checks the invoices table still says Pending Approval. Refusal, reason, and nothing changed."

After the pass: `cd ..`. Say: "One passed."

Segment 7: The feature in the UI, live (6:30 - 7:45)

Start a fresh server and open the browser:

```
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
```

Open http://localhost:5000. All passwords are `ChangeMe123!`.

Step	Action	What the viewer sees	Say
1	Sign in `demo.manager`, Approvals, Approve INV-2001 (75,000.00)	Red refusal text, invoice still listed	"Manager over their limit: refused with a reason, nothing changed."
2	Sign out, sign in `demo.director`, Approve INV-2001	Disappears from pending, becomes Approved	"A Director can approve it alone."
3	Still `demo.director`, Approve INV-2002 (600,000.00)	"1 of 2 approvals", still pending	"Above five hundred thousand the first approval is not enough."
4	Approve INV-2002 again	Refusal: already approved	"The same user cannot be both approvers."
5	Sign out, sign in `demo.director2`, Approve INV-2002	Approved	"A different Director completes it."

Stop the server with Ctrl+C. (Optional extra if time remains: sign in as `demo.manager` after step 3 on a fresh server and show a Manager refused as the second approver.)

Segment 8: PR review and verdicts (7:45 - 8:45)

Show: the pull request description on GitHub (or `docs/pr-review.md`), scrolling the findings table.

Say: "I reviewed my own diff with the pr-reviewer agent and judged each finding. Eleven findings, every one with exactly one verdict. Six fixed, three dismissed with reasons, two deferred with reasons and a tracking id. The two I want to highlight were both real bugs I would not have found by reading my own code. F1: two simultaneous approvals by the same Director both passed the already-approved check, which breaks the central rule. I fixed it with a lock, and wrote tests with a delayed-save repository that fail when I remove the lock. My first version of those race tests passed even without the lock, so they proved nothing; I caught that and rewrote them. F2: an amount like one-e-nine-nine-nine-nine-nine-nine was accepted and then broke the invoice list for everyone. Fixed with input validation. Dismissed means I wrote

down why, for example role names are matched exactly so a mistyped role fails closed. Deferred findings point at tasks T024 and T025."

Optional live proof for F2 (15 seconds): with the server running, in a second terminal sign in and post the huge amount; it must return 400 with a message and not break the list. Skip if short on time.

Segment 9: AI log and wrap-up (8:45 - 9:30)

Run:

```
Get-Content logs\ai-interactions.jsonl -Tail 6
(Select-String -Path logs\ai-interactions.jsonl -Pattern '"event": "CheckSummary"').Count
```

Say: "All AI interaction is logged by hooks into one file, one JSON object per line with the same 27 keys every time: the tool and model, the agent and its mode, the prompt, skills, sub-agents and tools used, what the AI suggested, the human decision, how it was verified, the status and the files touched. Test runs become CheckSummary lines with pass or fail. You can see the deliberate failure from earlier, and the green run after. The first few lines are marked reconstructed, because they cover planning before the hooks existed. To wrap up: the feature is in, with unit tests for every rule including both negative rules, mutation score from sixty-four to one hundred percent, one Playwright test, a reviewed PR with every finding judged, and the write-up in ASSIGNMENT.md. One honest limit: the project agents were not selectable by name in the session that created them, so three roles ran as general-purpose sub-agents following the agent files. Thanks for watching."

4. If something goes wrong on camera

Problem	Do this
Mutation run takes much longer than 20 seconds	Keep talking through the survivors story; if over 60 seconds, cut and show `docs/mutation/after.txt`, saying so.
WSL path error from Git Bash	Use PowerShell, or prefix `MSYS_NO_PATHCONV=1`
The manual walkthrough finds INV-2001 already approved	An earlier run approved it. Restart the server to reset the data. (The Playwright test does not depend on it.)
Port 5000 busy	`Get-NetTCPConnection -LocalPort 5000` then stop that process
You forgot to restore the file after the break demo	`git restore ...` or copy the backup back, then rerun pytest on camera: it is another proof
Over 10 minutes	Cut segment 3 to 15 seconds and skip the optional extras
Under 8 minutes	Add the optional Manager-as-second-approver step and the F2 curl proof

5. After recording

1. Check the video plays with sound and lasts 8 to 10 minutes.
2. Upload it (Teams recording link, or any host that gives a shareable link) and make sure the link opens without a sign-in problem for the reviewer.
3. Paste the link into `ASSIGNMENT.md` (replace the TODO line near the top) and rebuild the PDFs: `python scripts/build_docs.py`.
4. Commit and push (see the commit plan in `ASSIGNMENT.md`).